МІністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт Про з лабораторної роботи №7
з дисципліни “Основи програмування”
“ФУНКЦІЇ”

Виконав:
студент Феп-13
Тутов Максим

Перевірил:
ас.Васюта В.М

Львів 2026

Мета:Вивчити поняття і застосування функцій.
Обладнання та програмне забезпечення:
– IBM сумісна персональна обчислювальна машина;
– онлайн компілятор мови програмування Сі, доступний за посиланням https://repl.it/languages/c

Хід роботи

Завдання 1:Створити функцію, яка перевіряє, чи введене з консолі число є простим

```c
#include <stdio.h>

// Функція для перевірки, чи є число простим
int is_prime(int n) {
    if (n <= 1) return 0; // Числа <= 1 не є простими
    if (n <= 3) return 1; // 2 та 3 — прості числа
    if (n % 2 == 0 || n % 3 == 0) return 0;

    // Перевірка дільників з кроком 6 (оптимізований алгоритм)
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return 0;
        }
    }
    return 1;
}

int main() {
    int num;

    printf("Введіть ціле число для перевірки: ");
    if (scanf("%d", &num) == 1) {
        if (is_prime(num)) {
            printf("Результат: Число %d є простим.\n", num);
        } else {
            printf("Результат: Число %d НЕ є простим.\n", num);
        }
    } else {
        printf("Помилка: введено не коректне число.\n");
    }

    return 0;
}
```

```
Введіть ціле число для перевірки: 33
Результат: Число 33 НЕ є простим.
```

У ході виконання завдання було розроблено функцію is_prime, яка перевіряє введене з консолі число на просте. Алгоритм коректно відсікає числа менші або рівні 1, а також використовує оптимізовану перевірку дільників для визначення результату .

Завдання 2:Написати програму, котра рекурсивно вираховує суму чисел від Вашого
порядкового номера в журналі до 100.

```c
#include <stdio.h>

// Рекурсивна функція для обчислення суми чисел від current до target
int recursive_sum(int current, int target) {
    // Базовий випадок: якщо поточне число перевищує кінцеве, повертаємо 0
    if (current > target) {
        return 0;
    }
    // Рекурсивний випадок: додаємо поточне число до суми решти чисел
    return current + recursive_sum(current + 1, target);
}

int main() {
    int N = 12;
    int target = 100;

    // Перевірка на коректність діапазону
    if (N > target) {
        printf("Помилка: порядковий номер (%d) не може бути більшим за 100.\n", N);
        return 1;
    }

    // Виклик рекурсивної функції
    int total_sum = recursive_sum(N, target);

    // Виведення результату
    printf("Порядковий номер у журналі : %d\n", N);
    printf("Кінцеве число: %d\n", target);
    printf("Сума чисел від %d до %d дорівнює: %d\n", N, target, total_sum);

    return 0;
}
```

```
Порядковий номер у журналі : 12
Кінцеве число: 100
Сума чисел від 12 до 100 дорівнює: 4984
```

У цьому завданні було реалізовано рекурсивну функцію recursive_sum, яка обчислює суму чисел від заданого порядкового номера у журналі ($N = 12$) до 100 включно

Завдання 3:Написати програму, котра знаходить розв’язок рівняння N*x + A = 0 методом половинного ділення. N – Ваш порядковий номер в журналі, A – Ваш вік (повних років). Розв’язок знаходиться в інтервалі [–100..100]

main.c | Clear | Run ▶

```c
#include <stdio.h>
#include <math.h>

// Функція лінійного рівняння: f(x) = N * x + A
double f(double x, double N, double A) {
    return N * x + A;
}

// Метод половинного ділення для знаходження кореня f(x) = 0
double bisection_method(double N, double A, double a, double b, double eps) {
    // Перевірка умови існування кореня на інтервалі [a, b]
    if (f(a, N, A) * f(b, N, A) >= 0) {
        printf("Помилка: на заданому інтервалі немає кореня або функція не змінює знак.\n");
        return 0.0;
    }

    double c = a;
    while ((b - a) / 2.0 > eps) {
        c = (a + b) / 2.0;

        if (f(c, N, A) == 0.0) {
            break;
        }

        else if (f(a, N, A) * f(c, N, A) < 0) {
            b = c;
        } else {
            a = c;
        }
    }
    return (a + b) / 2.0;
}

int main() {
    // Вхідні дані
    double N = 12;      // Порядковий номер у журналі
    double A = 17;      // Вік (повних років)

    double a = -100.0;  // Ліва межа інтервалу
    double b = 100.0;   // Права межа інтервалу
    double eps = 1e-6;  // Точність обчислень (10^-6)

    printf("Розв'язання рівняння %.0f*x + %.0f = 0 методом половинного ділення\n", N, A);
    printf("Інтервал: [%.0f, %.0f]\n", a, b);

    double root = bisection_method(N, A, a, b, eps);

    printf("Знайдений корінь x = %.6f\n", root);


    printf("Перевірка (аналітичний розв'язок -A/N) = %.6f\n", -A / N);

    return 0;
}
```

Output

```
Розв'язання рівняння 12*x + 17 = 0 методом половинного ділення
Інтервал: [-100, 100]
Знайдений корінь x = -1.416666
Перевірка (аналітичний розв'язок -A/N) = -1.416667
```

Програма успішно застосувала чисельний метод дихотомії (половинного ділення) із заданою точністю, визначивши значення невідомої змінної x, що задовольняє умові рівняння.

Висновок:У ході виконання роботи було досліджено метод половинного ділення.
Також закріплено практичні навички:
ознайомлено з механізмами передачі аргументів у функції та організації повернення значень.
Реалізовано алгоритми з використанням умовних операторів і циклів для перевірки простих чисел.
Закріплено навички роботи з рекурсивними викликами функцій на прикладі обчислення суми послідовності чисел від заданого номера до 100.
Застосовано чисельні методи (метод половинного ділення) для знаходження коренів рівнянь з урахуванням індивідуальних параметрів (порядкового номера у журналі та віку) на заданому числовому інтервалі.